From Choices to Sums

Connections to OpenAI's Subset-Sum Algorithms

Jonathan R. Landers October 28, 2026

A list of n numbers has 2^n subsets. Different subsets can add to the same total. Recording each distinct total once can save repeated work, and a table of reachable sums can be built once and reused to check many target values. These two ways of saving work connect the repository's experiments to OpenAI's recent subset-sum manuscripts.

An exponential search with only two questions

Subset sum asks whether a subset of positive integer weights $w_1, \ldots, w_n$ adds to a nonnegative integer target t. Each item can be used once; equal values remain separate items.

Set aside 6 from $[2, 5, 6, 11, 23]$. The remaining *core*, $[2, 5, 11, 23]$, is *superincreasing*: each weight exceeds the sum of its predecessors. Test a target by working downward, taking each weight that fits and subtracting it. Target 17 fails without 6; taking 6 leaves 11, which the core reaches. Two core tests resolve the original 32 choices.

Note V [1] finds a largest such core by sorting and keeping weights that exceed the total already kept. This takes $O(n \log(n+1))$ arithmetic and comparison operations. Here $O(f)$ bounds growth by a constant multiple of f for sufficiently large inputs.

Let C contain the core indices and E the remaining indices, with $e = |E|$ exceptions. For any index set V, $|V|$ counts its items and its distinct subset sums are

$$\mathcal{A}(V) = \left\{ \sum_{i \in A} w_i : A \subseteq V \right\}.$$

This set is the *support*. Put $K = |\mathcal{A}(E)|$. Equal exceptional sums s leave the same core target $t - s$. Build the support from $\{0\}$, merging the current sorted list with a copy increased by each next weight. Keep one *witness* per sum: the indices of a subset producing it. Each intermediate list has at most K entries, since later items can be omitted. Construction takes $O(eK)$ operations and core tests take $O(nK)$.

Let L be the total number of bits encoding the weights and target, and let $\mathrm{poly}(L)$ denote an unspecified polynomial in L. Including integer arithmetic gives a bit-operation bound of $\mathrm{poly}(L)K$.

A shared measure of compression

OpenAI's low-space manuscript [2] measures a block of indices V through its *deficiency*,

$$d(V) = |V| - \log_2 |\mathcal{A}(V)|.$$

The logarithm is base two. Thus $K = 2^{e - d(E)}$, and Note V's runtime T_V satisfies

$$\boxed{T_V \le \mathrm{poly}(L)\, 2^{e - d(E)}.}$$

The formula identifies two sources of savings: fewer exceptional items and more merging among their sums. Twelve equal positive exceptions have only 13 sums, giving $d(E) = 12 - \log_2 13$. Their 4,096 selections require just 13 core tests. The exponent is the base-two logarithm of the number of distinct residual targets.

Few sums compress; many sums support filtering

The low-space manuscript runs two randomized trials [2]. One merges repeated sums in disjoint blocks and generates combined sums in sorted order. The other uses blocks with many distinct sums to give a solution many *representations*: different splits into partial subsets. It filters records by chosen modular remainders; enough representations allow the solution to survive. The proof analyzes both trials using the average deficiency of a randomly chosen block.

These complementary uses of distinct sums follow earlier work by Austrin, Kaski, Koivisto and Nederlof [3].

Let b be the largest bit length of a weight or target. *Polynomial-bit* means $b \le n^c$ for a fixed constant $c > 0$. Using $O(n + b)$-bit machine words, the September manuscript states $\mathrm{poly}(n)2^{n/2}$ time and $O(2^{n/5})$ writable words of memory. Every YES answer is correct; each solvable input is accepted with probability at least $2/3$ over the algorithm's random choices [2].

Overlapping subsets need index checks. In $[2, 3, 5]$, a sum of 5 can use 5 or 2 and 3; only the first can combine with the item 2 to give 7. OpenAI retains *membership masks*: bits recording which shared indices a partial subset uses. Note V's core and exceptions share no indices, so one witness per sum suffices. Merge records when later combinations treat them identically.

A checksum that recovers the exact target

The October manuscript [4] states $O(2^{0.49n})$ word operations for polynomial-bit inputs. This time bound holds on every execution, and each fixed input receives the correct answer with probability at least $2/3$. Classical *meet-in-the-middle* splits the weights in half, lists each half's subset sums and searches for a pair adding to the target. Its $2^{n/2}$ exponential factor is larger than $2^{0.49n}$ by a factor of $2^{0.01n}$.

A preliminary test handles small partial-sum supports. Random tie-breaking then produces transformed weights c_i and targets t'; on a solvable input, with at least a fixed positive probability one target has exactly one solution. For an index subset I, let $c(I) = \sum_{i \in I} c_i$, and let $\chi(I)$ be its assigned random complex number of absolute value one, called a *phase*. Choose a prime P. The notation $c(I) \equiv t' \pmod P$ means the two numbers have the same remainder on division by P. An *alias* matches that remainder while having a different total. Summing over index subsets gives

$$\underbrace{\sum_{c(I)=t'} \chi(I)}_{\text{exact target}} = \underbrace{\sum_{c(I)\equiv t' \pmod P} \chi(I)}_{\text{modular checksum}} - \underbrace{\sum_{\substack{c(I)\equiv t' \pmod P \\ c(I)\neq t'}} \chi(I)}_{\text{aliases}}.$$

A Fourier formula rewrites the modular checksum as an average of products built from individual weights. Filters organize which terms to evaluate or sample, and compatible partial records recover the aliases. The difference is zero for an unreachable target and has absolute value one for a unique solution. Computing it to absolute error below $1/4$ lets the algorithm distinguish these cases by comparing its absolute value with $1/2$ [4].

A geometric representation that answers many targets

Note VII [5] builds a table of reachable points. After exact preprocessing, write the remaining instance as positive weights $u_1, \ldots, u_n$ and nonnegative target T; L is its binary input length. Choose an integer base $M \geq 2$ and write

$$u_i = Mp_i + r_i, \qquad p_i \geq 0,$$

where p_i is an integer quotient and r_i is a *residual*, a signed remainder. Select core indices C whose residuals have a small total range, and put the remaining indices in E, with $e = |E|$. Let $x_i = 1$ when core item i is selected and $x_i = 0$ otherwise. The selection has total quotient q and residual sum s:

$$(q, s) = \left(\sum_{i \in C} p_i x_i,\ \sum_{i \in C} r_i x_i \right).$$

Starting at $(0, 0)$, process each core item by keeping the current points and also adding copies shifted by (p_i, r_i). Selecting an exceptional subset $A \subseteq E$ leaves the core target z_A, giving the line

$$Mq + s = z_A, \qquad z_A = T - \sum_{i \in A} u_i.$$

The same reachable points serve every shifted target. With core $[101, 203]$, exception 149, base $M = 100$ and target 250, taking the exception shifts the line to $100q + s = 101$. The reachable point $(1, 1)$ supplies the core item 101. The original solution is $149 + 101 = 250$.

Let R_- and R_+ bound the residual sum of any core subset. The table has residual width W and quotient height H, with largest stored quotient Q:

$$R_- = \sum_{i \in C} \min(0, r_i), \quad R_+ = \sum_{i \in C} \max(0, r_i), \quad W = R_+ - R_- + 1,$$

$$Q = \min\left(\sum_{i \in C} p_i, \left\lfloor \frac{T - R_-}{M} \right\rfloor \right), \qquad H = Q + 1.$$

Here $\lfloor y \rfloor$ means rounding y down to an integer. Positive exceptional weights give $z_A \leq T$, so every solution has $Mq = z_A - s \leq T - R_-$. The nonnegative p_i make it safe to discard points above Q while building the table. Each candidate integer q requires one test of $(q, z_A - Mq)$. Let G, with $1 \leq G \leq H$, bound the number of such tests per target.

Let F be the bit-operation cost of preprocessing and discovering the base and core. The implementation stores each quotient row in a W-bit integer; a row lookup takes at most $O(W)$ bit operations. The runtime companion [6] therefore bounds the total runtime T_{geom} by

$$T_{\text{geom}} \leq F + O^*(HW + 2^e GW),$$

where O^* hides factors polynomial in L. Construction costs $O^*(HW)$; testing the 2^e exceptional subsets costs $O^*(2^e GW)$. One table can represent exponentially many core subsets. If F and HW grow polynomially in L, and $e = O(\log L)$, the total runtime is polynomial.

Combining the two structural savings

Apply Note V's support construction to the geometric exceptions E, using the reduced weights u_i. Keep one witness per distinct sum and test each residual target against the shared table. Define the exceptional support $\mathcal{A}(E)$, its size K and its deficiency $d(E)$ as before.

Derived connection. Combining Note V's support construction with Note VII's shared table gives an exact algorithm whose runtime T_{combined} satisfies

$$\boxed{T_{\text{combined}} \le F + O^*\Big(HW + 2^{e-d(E)}(1+GW)\Big).}$$

The support construction costs $O^*(K)$; bounds and table queries cost $O^*(K(1+GW))$: the 1 pays for checking each target's bounds, and GW pays for table tests. One core table costs $O^*(HW)$. Saving intermediate tables to reconstruct a core subset, together with exceptional witnesses, occupies $O^*(HW+K)$ bits.

For polynomial-bit inputs, choose a fixed exponent margin $0 < \epsilon < 0.49$. If F, HW and $K(1+GW)$ are each $O^*(2^{(0.49-\epsilon)n})$, the combined runtime grows more slowly than $2^{0.49n}$. The fixed margin ϵ absorbs the polynomial factors hidden by O^*.

Note III [7] also uses repeated sums to count reachable targets. Split the weights into two disjoint groups with distinct-sum sets S_1, S_2. Fix a finite target set D and let $r(t) = |\{(a,b) \in S_1 \times S_2 : a+b=t\}|$ count the sum pairs reaching t. Define

$$B = \sum_{t\in D} r(t), \qquad J = \sum_{t\in D} \binom{r(t)}{2}.$$

B counts representations and J counts unordered pairs of representations with the same target. If N targets in D are reachable, the identity $\sum_{t\in D} r(t)^2 = B + 2J$ and Cauchy–Schwarz give $B^2 \le N(B+2J)$. Thus, for $B > 0$, at least $B^2/(B+2J)$ distinct targets are reachable. Representation counts and their collisions together bound how much of the target set is covered.

The experiments charge for finding and using structure. In Note V, two procedures chose cores leaving fewer distinct sums on 18 of 54 structured lists; their median total times were about nine times the simple core solver's. In Note VII, reusable cores raised completed exact decisions from 453 to 469 of 576 fresh constructed targets under the same work allowance. Resampling whole weight lists gave a 95% bootstrap interval of $[4, 30]$ for the net gain. A later comparison with discovery fixed avoided 90 of 182 table builds by first rejecting shifted targets through numerical bounds.

OpenAI's worst-case bounds and the repository's structural algorithms both exploit repeated sums and carefully chosen records. Their costs lead to the question: *how much subset search can a compact record replace, after paying to build and use that record?*

References

[1] J. R. Landers. *How Far from Easy? Structural Distance in Subset Sum*. Note V, working draft, 2026.
[2] OpenAI. *A Low-Space Algorithm for Worst-Case Subset Sum*. Preprint, September 26, 2026.
[3] P. Austrin, P. Kaski, M. Koivisto and J. Nederlof. Dense Subset Sum May Be the Hardest. *STACS*, 13:1–13:14, 2016.
[4] OpenAI. *Subset Sum in Time $O(2^{0.49n})$*. Preprint, October 4, 2026.
[5] J. R. Landers. *Geometric Cores and Exceptions in Subset Sum*. Note VII, working draft, 2026.
[6] J. R. Landers. *The Runtime of Geometric Core Subset Sum*. Runtime companion to Note VII, working draft, 2026.
[7] J. R. Landers. *Probability-Valued Subset Sum: Certified Bounds and Learned Guidance*. Note III, 2026.